13.06 Text files

Data need to be stored permanently. One approach is to use a file. For example, any data held in an array while your program is executing will be lost when the program stops. You can save the data out to a file and read it back in when your program requires it on subsequent executions.

A text file consists of a sequence of characters formatted into lines. Each line is terminated by an end-of-line marker. The text file is terminated by an end-of-file marker.



TIP

You can check the contents of a text file (or even create a text file required by a program) by using a text editor such as NotePad.

Writing to a text file

Writing to a text file usually means creating a text file.

The following pseudocode statements provide facilities for writing to a file:

```
OPENFILE <filename> FOR WRITE        // open the file for writing
WRITEFILE <filename>, <stringValue>  // write a line of text to the file
CLOSEFILE <filename>                 // close file
```

Reading from a text file

An existing file can be read by a program. The following pseudocode statements provide facilities for reading from a file:

```
OPENFILE <filename> FOR READ          // open file for reading
READFILE <filename>, <stringVariable>  // read a line of text from the file
CLOSEFILE <filename>                  // close file
```

Appending to a text file

Sometimes we may wish to add data to an existing file rather than create a new file. This can be done in Append mode. It adds the new data to the end of the existing file.

The following pseudocode statements provide facilities for appending to a file:

```
OPENFILE <filename> FOR APPEND      // open file for append
WRITEFILE <filename>, <stringValue> // write a line of text to the file
CLOSEFILE <filename>                // close file
```

The end-of-file (EOF) marker

If we want to read a file from beginning to end, we can use a conditional loop. Text files contain a special marker at the end of the file that we can test for. Testing for this special end-of-file marker is a standard function in many programming languages. Every time this function is called it will test for this marker. The function will return FALSE if the end of the file is not yet reached and will return TRUE if the end-of-file marker has been reached.

In pseudocode we call this function `EOF()`. We can use the construct `REPEAT...UNTIL EOF()`. If it is possible that the file contains no data, it is better to use the construct `WHILE NOT EOF()`.

For example, the following pseudocode statements read a text file and output its contents:

```
OPENFILE "Test.txt" FOR READ
WHILE NOT EOF("Test.txt") DO
```

```
        READFILE "Test.txt", TextString
        OUTPUT TextString
    ENDWHILE
    CLOSEFILE "Test.txt"
```

TASK 13.08

1 Write pseudocode to save the array data from Task 13.06 to a text file.

2 Write pseudocode to read the values stored in the text file back into the board array.

13.07 Abstract Data Types (ADTs)

An **Abstract Data Type** is a collection of data and a set of associated operations:

- create a new instance of the data structure
- find an element in the data structure
- insert a new element into the data structure
- delete an element from the data structure
- access all elements stored in the data structure in a systematic manner.

KEY TERM

Abstract Data Type: a collection of data with associated operations

The remainder of this chapter describes the following ADTs: stack, queue and linked list. It also demonstrates how they can be implemented from arrays.

In the following ADTs data items are represented as a single character, but this would normally be a set of data, possibly stored as fields in a record.

13.08 Stacks

What are the features of a stack in the real world? To make a stack, we pile items on top of each other. The item that is accessible is the one on top of the stack. If we try to find an item in the stack and take it out, we are likely to cause the pile of items to collapse.

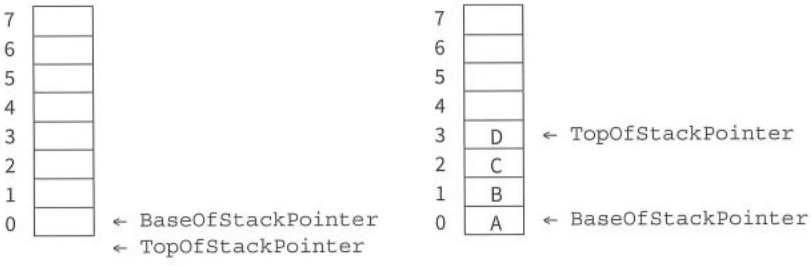


Figure 13.07 An empty stack (left) and a stack with four items pushed (right)